

《软件工程综合实践I》课程项目任务说明书

(4天集中实训版)

一、课程基本信息

- 课程名称：软件工程综合实践
- 授课对象：软件工程专业 大三学生
- 课时安排：共 4 天，32 课时（每天 8 课时）
- 结课方式：第 4 天下午项目答辩 + 代码评审
- 核心工具：Claude Code（AI 辅助编程）+ Docker + GitHub

二、项目目标

本项目要求学生在 **AI 辅助编程工具（Claude Code）** 的协助下，以小组形式独立完成一个具有实际业务场景、集成大模型智能功能、可容器化部署的完整 Web 应用系统。通过本项目，学生应达到以下能力：

- 掌握基于 **Spring Boot + MybatisPlus + Maven** 的后端开发全流程；
- 掌握 **Vue 3 / React 18** 前端框架与后端 API 联调；
- 掌握使用 **Docker + Docker Compose** 进行容器化部署与本地持续集成；
- 掌握调用**外部大模型 API** 构建智能功能的方法（非简单聊天机器人）；
- 理解并实践软件工程规范：Git 版本控制、数据库设计、API 文档、错误处理等；
- 体验并评估 **AI 辅助编程** 对开发效率与代码质量的影响。

三、必选技术栈（强制要求）

类别	技术选型	版本/说明
后端核心	Java + Spring Boot + Maven + MybatisPlus	Java 17, Spring Boot 3.x, Maven 3.8+, MybatisPlus 3.5.x
数据库	MySQL 8.0 或 PostgreSQL 14+（二选一）	强制，需提供 DDL
前端	Vue 3 全家桶（推荐）或 React 18 + Tailwind CSS	二选一
API 文档	SpringDoc OpenAPI (Swagger)	自动生成接口文档
版本控制	Git + GitHub	代码必须托管于公共或私有仓库
容器化与部署	Docker + Docker Compose	提供一键运行环境
AI 辅助工具	Claude Code（CLI 或 Web 版）	用于代码生成、调试、重构、测试
智能模块	调用任意外部大模型 API（Claude / OpenAI / 智谱 / 通义千问等）	必须与业务场景结合

四、项目功能要求

4.1 业务功能（最低标准）

每个项目必须包含至少 3 个主要业务模块（例如：管理端 + 用户端 + 统计/分析），并能演示完整业务流程。推荐选题方向（任选其一或自行设计，但需经教师同意）：

- 在线考试系统：学生考试、教师出卷、自动阅卷/成绩分析。
- 图书借阅管理系统：用户借阅、图书推荐、逾期统计与罚款。
- 课程作业互评系统：提交作业、匿名打分、评语情感分析。
- 宿舍报修系统：报修申请、维修派单、满意度评价。
- 个人博客/内容平台：文章发布、评论、AI 生成摘要或标签推荐。
- 其他：具有明确角色（管理员/普通用户）及完整业务流程的系统。

4.2 AI 模块（强制，分值占比 25%）

每个项目必须包含至少一个真实调用外部大模型 API 的智能功能，且该功能必须与业务有机结合。纯粹的通用聊天机器人不计入核心 AI 模块。

✔ 可采纳方向（示例，不限于此）

- 智能内容生成：自动生成试题、作业报告的摘要、代码注释建议、文章标题等。
- 智能推荐/分析：图书推荐（基于历史借阅）、异常日志智能诊断、用户评价情感分析（正面/负面）。
- 自然语言查询（NL2SQL）：用户输入自然语言问题，系统转换成 SQL 并返回数据（如“查询上月销量最高的商品”）。
- 对话式辅助：智能客服解答系统使用问题，且答案需结合当前系统的业务数据。

🔧 技术约束

- 调用必须由后端（Spring Boot）发起，前端仅负责收集用户输入和展示结果。
- 必须处理 超时、重试、错误降级（例如 API 不可用时返回固定兜底文案，不得直接抛出 500 错误）。
- 需在 docs/ai_module.md 中说明：
 - 使用的模型名称、API 提供方、请求/响应示例；
 - 提示词（Prompt）设计思路及截图；
 - 调用流程图（可手绘拍照或使用 draw.io）；
 - 如何控制成本/限流（例如仅允许每人每天 10 次调用，或使用低配额测试 Key）。

4.3 非功能性要求

- 后端：所有接口遵循 RESTful 风格，Swagger 文档可访问；实现统一异常处理（@RestControllerAdvice）；参数校验使用 Jakarta Validation（@Valid）。
- 前端：实现路由守卫（基于登录状态和角色）；包含 loading 状态、错误提示等基础交互反馈；界面使用 Tailwind CSS 保证基本整洁。

五、数据库设计强制约束

为防止项目过于简单，强制执行以下数据表复杂度要求：

约束项	具体要求	验证方式
最小业务表数量	至少 5 张（不包括中间表、日志表、字典表、临时表）	检查 DDL 及 ER 图
必须提交的文档	提供 ER 图（图片/png/draw.io 链接）+ 完整 DDL 文件 <code>sql/schema.sql</code>	文档与代码完全一致

六、容器化与部署要求

- 项目根目录必须包含：
 - 后端 `Dockerfile`（基于 `openjdk:17-jdk-slim`，多阶段构建可选）；
 - 前端 `Dockerfile`（使用 `node:18` 构建 + `nginx:alpine` 提供静态资源）；
 - `docker-compose.yml`。
- `docker-compose.yml` 必须定义三个服务：`backend`、`frontend`、`db`（MySQL 8.0 或 PostgreSQL 14+）。
- 容器间网络互通，后端通过服务名连接数据库（例如 `jdbc:mysql://db:3306/app_db`，不允许使用 `localhost` 或 IP）。
- 使用 `volumes` 持久化数据库数据，防止容器重启后数据丢失。
- 最终验收标准：教师或助教仅需执行一条命令 `docker compose up --build`，即可完整运行整个系统，并能通过 `http://localhost` 访问前端界面、通过 `http://localhost:8080/swagger-ui/index.html` 访问 API 文档。

七、版本控制与 AI 使用记录

7.1 GitHub 仓库要求

- 每组创建独立 GitHub 仓库或者Gitee仓库（公开或私有）。
- 整个项目周期内 至少 8次有意义的 `commit`，提交信息应清晰反映变更内容（禁止仅写“update”）。

7.2 Claude Code 对话记录提交（强制）

为什么提交：用于评估学生与 AI 协作的真实过程，避免直接粘贴代码而不理解。

提交格式：将关键会话导出为 **Markdown 文件**，存放于 `docs/claude_sessions/` 目录，文件名按内容命名，如 `day1_generate_mapper.md`、`day2_debug_docker.md`。

导出方法（任选其一）：

- 通过对话方式导出对话：如“帮我将当前对话导出成md文档”。
 - 也可以手动复制对话内容（但务必保留完整上下文，不得只截取 AI 的回答）。
- 最低要求：**
- 每组至少提交 **3 次** 有意义的 AI 协作对话，内容需覆盖以下不同阶段（至少三项）：
 - 生成 MybatisPlus Mapper/Service 代码
 - 调试后端错误（如 SQL 异常、依赖冲突）
 - 编写 Dockerfile 或 docker-compose.yml
 - 集成大模型 API（AI 模块）
 - 前端 API 调用代码生成
 - 对话记录必须体现学生与 AI 的**交互过程**（提问 → 回答 → 追问/修改 → 最终确认），而不是单次“给我代码”就结束。

八、4 天冲刺进度安排（32 课时）

说明：每天 8 课时（上午 4 课时 + 下午 4 课时），第 4 天下午为答辩。所有小组必须按以下里程碑当天提交到 GitHub。

天数	时段	主题与任务	里程碑产出（必须当天提交到 GitHub，截止当天 23:59）
第 1 天	上午 (4 课时)	环境搭建 + 项目初始化 + 数据库设计 1. 安装 Java 17, Maven, Git, Docker Desktop, IDE，配置 Claude Code。 2. 创建 GitHub 仓库，初始化 Spring Boot + MybatisPlus 项目。 3. 配置 <code>application.yml</code> 连接本地数据库。 4. 设计 ER 图，确定至少 5 张表及关系。	1. GitHub 仓库链接 2. ER 图文件 <code>docs/database_design.md</code> 3. DDL 脚本 <code>sql/schema.sql</code>
	下午 (4 课时)	后端 CRUD 开发 (AI 辅助) 1. 使用 Claude Code 生成 Entity, Mapper, Service, Controller。 2. 完成至少 3 张核心表的 CRUD API。 3. 通过 Postman 或 Swagger 测试接口。 4. 实现全局异常处理和参数校验。	1. 完整的后端 CRUD 代码（可运行） 2. Swagger 文档可访问 3. 1 次 Claude Code 对话记录（生成 Mapper/Service）
第 2 天	上午 (4 课时)	前端脚手架 + 登录/鉴权 1. 初始化 Vue 3 或 React 项目，安装 Tailwind CSS，配置路由。 2. 实现登录/注册页面。 3. 后端集成 JWT（或使用 Session + Spring Security 简化版）。 4. 前端调用登录接口并存储 token。	1. 前端可以成功登录并跳转主页 2. 提交前端代码及后端认证代码 3. 数据库至少有一条测试用户记录
	下午 (4 课时)	剩余业务模块 + 前后端联调 1. 开发剩余 2 个业务模块的前端页面。 2. 前端调用后端 API 实现列表展示、分页、表单提交。 3. 确保所有预设 API 与前端完成数据流动。 4. 处理跨域（CORS）问题。	1. 主要业务流程可以手动走通（从登录到核心功能） 2. 提交全部前后端代码 3. 第 2 次 Claude Code 对话记录（如前端 API 调用代码生成）
第 3 天	上午 (4 课时)	AI 模块集成 1. 后端调用外部大模型 API（申请免费试用 Key）。 2. 实现业务相关的 AI 功能（内容生成/推荐/NL2SQL 等）。 3. 增加超时、重试、降级处理。 4. 前端添加 AI 功能入口并展示结果。	1. AI 功能可以独立测试（提供测试页面或 API 文档） 2. 提交 <code>docs/ai_module.md</code> 文档 3. 第 3 次 Claude Code 对话记录（AI 模块生成）
	下午 (4 课时)	容器化 + 整合测试 1. 编写后端 Dockerfile，前端 Dockerfile， <code>docker-compose.yml</code> 。 2. 本地执行 <code>docker compose up --build</code> 验证一键启动。 3. 修复容器化过程中的问题（数据库连接、端口映射、环境变量）。 4. 确保前后端与数据库容器间正常通信。	1. <code>docker-compose.yml</code> 可成功启动全部服务 2. 演示视频或现场验证： <code>localhost</code> 访问完整系统 3. 提交 Dockerfile 和 compose 文件
		项目完善 + 文档整理 + 答辩准备	

第4天	上午 (4课时)	1. 修复剩余 bug，优化 UI 细节。 2. 完善 README.md（必须包含：项目简介、技术栈、如何运行（docker compose up 即可）、AI 模块说明）。 3. 整理所有文档（数据库设计、AI 设计、对话记录）。 4. 准备 5 分钟答辩演示（PPT + 现场演示）。	1. 最终版代码推送到 main 分支 2. 提交所有必需文档（至少：README.md, docs/ai_module.md, docs/database_design.md, docs/claude_sessions/ 下 3 个文件） 3. 确保 main 分支代码可通过 docker compose up --build 一键运行
	下午 (4课时)	项目答辩与评审 1. 每组 5 分钟演示（展示业务功能 + AI 模块 + 容器化一键启动）。 2. 教师随机提问（数据库设计、AI 调用、AI 辅助开发过程、遇到的困难等）。 3. 根据评分标准现场打分。	1. 现场演示成功（不出现严重崩溃，如数据库连不上） 2. 每组提交答辩 PPT（可选，可用 README 代替） 3. 教师记录分数及评语

九、评分标准（总分 100 分）

评分项	分值	详细评分点
功能完整性	20	3 个业务模块均正常工作，业务流程完整，无明显 bug。
AI 模块实现	25	真实调用外部大模型 API，与业务场景深度结合；有错误处理与降级策略；文档说明清晰；演示效果好。
数据库设计	12	表数量 ≥5，关系类型 ≥3，有软删除和审计字段，提供 ER 图和 DDL，符合第三范式基本要求。
容器化与部署	12	docker compose up --build 无报错启动全部服务，所有功能在容器中正常运行；Dockerfile 合理分层。
代码质量与规范	8	命名规范，分层清晰，有异常处理和参数校验，Git 提交历史清晰（≥8次 commit，信息有意义）。
AI 对话记录	8	提交至少 3 次有效对话记录，内容完整，能体现与 AI 协作解决问题的迭代过程。
需求文档	5	内容完整，需求清晰，与最终实现一致。
开发文档	5	包含架构图、环境配置、项目结构，具有可操作性。
文档与答辩表现	5	README 清晰准确；数据库设计和 AI 设计文档完整；答辩演示流畅，能正确回答问题。

加分项（≤10 分）

- 前端实现高级交互，如 WebSocket 实时通知（+5 分）。
- 对 AI 模块或核心业务逻辑编写单元测试（+5 分）。

九、提交物清单（截止第 4 天中午 12:00）

所有文件必须推送到 GitHub 仓库的 main 分支，并保证目录结构清晰。

序号	提交物类别	具体内容与格式要求	存放路径/文件命名
1	源代码	后端（Spring Boot + MybatisPlus）完整代码，前端（Vue/React）完整代码	<code>/backend</code> , <code>/frontend</code>
2	容器化文件	<code>Dockerfile</code> （后端）, <code>Dockerfile</code> （前端）, <code>docker-compose.yml</code>	根目录 <code>/Dockerfile</code> （后端可放 <code>/backend/Dockerfile</code> , 前端同理；compose 放根目录）
3	数据库脚本	建表语句 <code>schema.sql</code> , 可选初始数据 <code>data.sql</code>	<code>/sql/schema.sql</code> , <code>/sql/data.sql</code>
4	需求文档	包含：项目背景、用户角色、功能性需求（用例图或列表）、非功能性需求、业务规则	<code>/docs/requirements.md</code> 或 <code>/docs/需求文档.md</code>
5	开发文档	包含：系统架构图（分层或微服务）、技术选型说明、环境配置步骤、项目结构说明、API 简要说明	<code>/docs/development.md</code> 或 <code>/docs/开发文档.md</code>
6	数据库设计文档	包含：ER 图（图片或可编辑链接）、表结构说明（字段含义、约束、索引）、关系说明	<code>/docs/database_design.md</code>
7	AI 模块设计文档	包含：使用的模型、提示词设计、调用流程图、降级策略、费用控制方法	<code>/docs/ai_module.md</code>
8	Claude Code 对话记录	至少 3 次关键会话，Markdown 格式，保留完整交互过程	<code>/docs/claude_sessions/*.md</code>
9	项目 README	包含：项目简介、小组分工、技术栈、如何运行（ <code>docker compose up</code> 即可）、AI 功能说明、常见问题	根目录 <code>/README.md</code>
10	答辩 PPT	5 分钟演示材料（可选，可用 README + 现场演示代替）	根目录 <code>/答辩.pptx</code> 或 <code>/答辩.pdf</code>

 **重要：**缺少需求文档或开发文档，总分扣除 10 分（各扣 5 分）。文档内容需与最终代码保持一致。

十一、学术诚信说明

- 允许在 Claude Code 等 AI 辅助工具帮助下完成代码，**但学生必须理解并能解释每一段关键代码**（尤其是 AI 生成的部分）。
- 严禁**直接复制粘贴 AI 生成的整个项目而不做任何修改。发现两组项目代码结构或数据库设计高度雷同，均视为作弊，按零分处理。
- 所有 AI 生成的代码需在 **commit message** 中标注，例如：`feat: add user login API (generated by Claude, then manually adjusted)`。未标注的 AI 代码若与另一组相似，按雷同处理。
- 答辩环节，教师有权随机要求某位组员解释任意模块的实现细节，若无法回答，小组该评分项降级处理。

祝各小组项目顺利，在 4 天中体验高效的 AI 辅助开发与工程化实践！